ME 6170 Project

John Gomes

University of Connecticut

Professor Paul Papas

Spring 2025

Overview

The goal of this experiment is to explore the influences of initial temperature and pressure on the final temperature, as well as the ignition delay time, τ. Plots of how the ignition delay time was calculated, and how it is affected by initial conditions will be shown below.

To model this, the Lagrangian Particle Simulation, provided by Cantera was used as the basis for this simulation [1]. From this, two cases were set up to vary the temperature in increments of 250 K, from 750 K to 2000 K. Also, the initial pressure was varied at 1, 5 and 10 atm. This was done for both a H2-O2 and CH4-O2 reaction. Both reactions were a 1% fuel volume at stoichiometric conditions and the remaining bath gas to nitrogen. See the Appendix for the code used.

Methods

The ignition delay time is calculated using the methodology shown in the figure below. Figure 1 is an example on a reduced scale to show better detail. Ignition delay time was solved for in the code using the maximum gradient along the temperature profile, along with point slope form to find the intersection between the tangent line and T_0. This intersection length, z_τ was then used along with velocity to get ignition delay time. Tables including all ignition delay times can also be found in the Appendix.

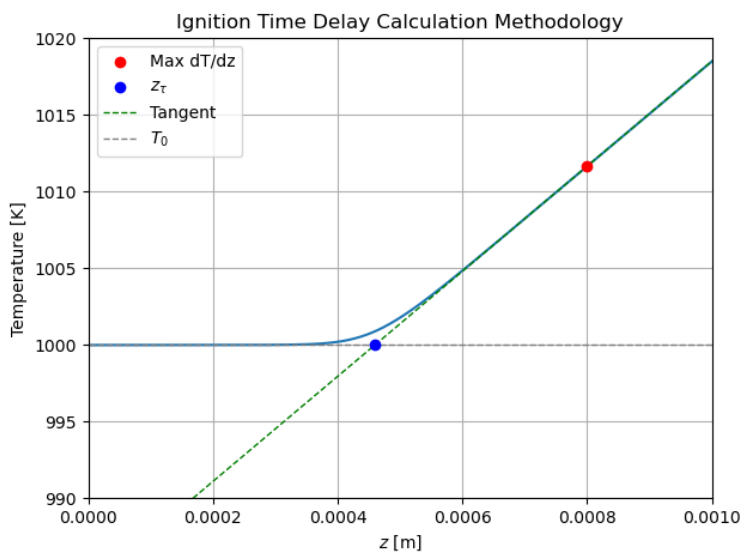

Figure 1: Ignition Time Delay Calculation Example

Analysis

See the figures below for a comparison of ignition delay time versus inlet temperature. This relationship is shown for all three studied pressures.

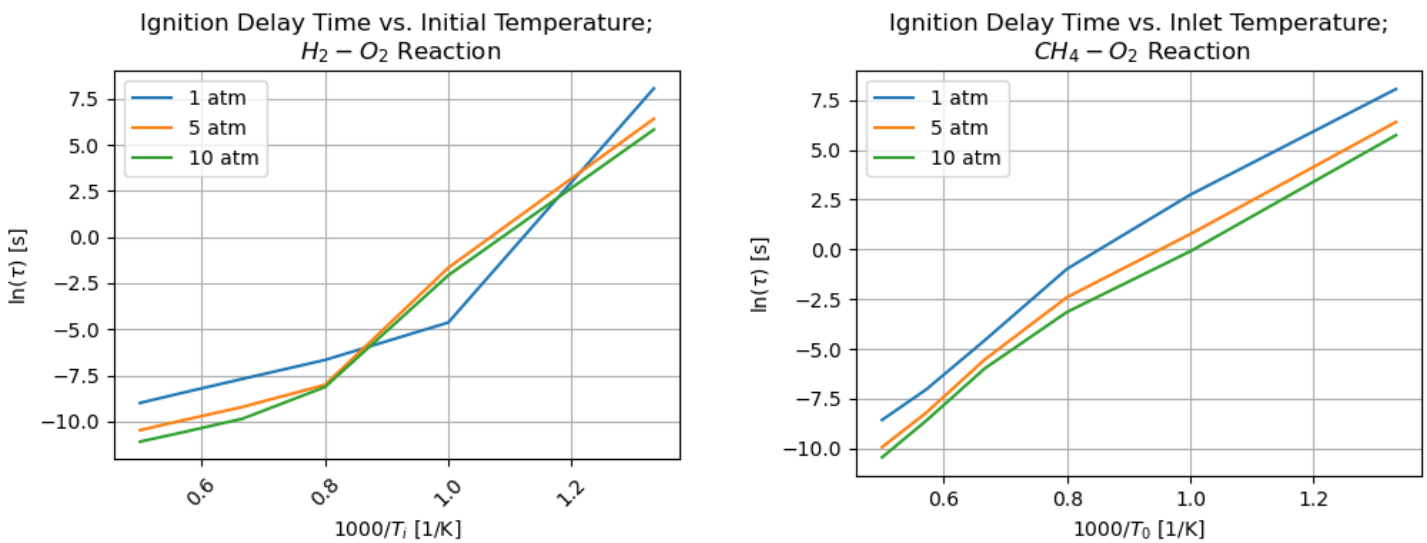

Figure 2: Ignition Time Delay and Initial Temperature

The slope of this plot is equal to E_a/R, where E_a is the activation energy, and R is the universal gas constant. From this, a larger slope means a higher activation energy. This is derived from Arrhenius kinetics [2].

$$\tau(T_f) = A * \exp\left(\frac{E_a}{RT_f}\right) \tag{1}$$

Visually, the slopes convey the reactions in a methane system are slower than that of the hydrogen system, as they have lower activation energy. Both plots are piecewise linear. Looking at the hydrogen system, at 1 atm, there are two points of inflection, 1000 K and 1250 K. For the methane system, a point of inflection at 1250 K. These separate high and low activation energies, like figure 2.3 in Glassman's text [2]. Simply, this represents a change in the rate-limiting step as the temperature changes.

See the table below for the activation energies of both systems, calculated by the slope. The limits correspond to the piecewise linear segments, reading right to left in figure 2 (low to high temperature).

Table 1: Derived Activation Energies

	H2-O2 E_a [kJ/mol]			CH4-O2 E_a [kJ/mol]	
Pressure [atm]	1st	2nd	3rd	1st	2nd
1	317.01	84.58	64.63	140.85	210.43
5	201.66	264.25	68.23	137.24	208.81
10	197. 22	252.25	82.18	138.48	202.52

The different rate-limiting steps at different temperatures can be determined by comparison to Glassman's book [2].

H2-O2 System:

1st Limiting Step	$H_2O_2 \rightleftharpoons OH + OH$	$[202.5\ \mathrm{kJ/mol}]$
2nd Limiting Step	$H_2 + M \rightleftharpoons H + H + M$	$[436.8\ \mathrm{kJ/mol}]$
3rd Limiting Step	$H + O_2 \rightleftharpoons O + OH$	$[69.5\ \mathrm{kJ/mol}]$

The activation energies at different temperatures align fairly well for the hydrogen system. At 1 atm, the 1st and 2nd activation energies are off, likely due to the limited number of temperature steps used in the code. The 2nd limiting step is the closest activation energy as provided by Glassman's book. The provided activation energy is likely the average of two rate-limiting steps. Disregarding this, the 5 and 10 atm simulations align closer with the activation energies for the 1st and 3rd limiting steps, provided by Glassman.

CH4-O2 System:

1st Limiting Step	$CH_3 + O_2 \rightleftharpoons CH_3O + O$	$[122.3\ \mathrm{kJ/mol}]$
2nd Limiting Step	$CH_2O + OH \rightleftharpoons HCO + HO_2$	$[217.6\ \mathrm{kJ/mol}]$

For the methane system, the activation energies are even closer to the text. There is less uncertainty in the methane plots, due to the stability of this system. Note that the Cantera mechanism file, 'gri30.yaml' uses hundreds of reactions. Therefore, it is undetermined if the steps provided by Glassman are truly the rate-limiting steps in this simulation, though it is likely.

While the rate-limiting step does not change with pressures studied, the ignition delay time. With this, the activation energy must also change, as it is a function of the inlet temperature and ignition delay time. Higher pressures result in faster chemical reactions due to an increased number of collisions for almost all cases studied. Comparison of these times can be found in the Appendix.

Conclusion

Through this study, the effects of pressure and temperature in a PFR simulation are now understood. Pressure has no effect on the rate-limiting step but does decrease the ignition delay time. Pressure also has no observed effect on the final temperature. Furthermore, the general behavior of the methane and hydrogen system is clear. Methane systems tend to have lower activation energy, independent of the inlet pressure and temperature. With this, the methane system also has longer ignition delay times across the board. These plots attest to the fact that hydrogen is a great fuel, with such a high release of energy. If this simulation was to be done again, a smaller temperature interval would be used to get more resolution on the plots. With this, the activation energies derived from the hydrogen system would better agree with the values provided in the text.

References

[1] D. G. Goodwin, H. K. Moffat, R. L. Speth, *Cantera: An Object-oriented Software Toolkit for Chemical Kinetics, Thermodynamics, and Transport Processes*, https://www.cantera.org, (Version 3.1.0).

[2] Glassman, I., & Yetter, R. A. (2014). *Combustion* (4th ed.). Elsevier Science.

Appendix

In [822…
```
import cantera as ct
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
np.set_printoptions(legacy='1.25')
```

H2-O2 Simulation

In [815…
```
### INPUTS
composition_0 = 'H2:0.01, O2:0.005, N2:0.985' #1% vol fuel, phi =1, rest N2
pressures = [1,5,10] # constant pressure [Pa]
lengths = [25, 0.001, 1.5e-5, 0.5e-5, 1.5e-5, 1.5e-6] #PFR Length
Temperatures=[750, 1000, 1250, 1500, 1750, 2000] # inlet temperature [K]
u_0 = .006  # inflow velocity [m/s]
area = 1.e-4  # cross-sectional area [m**2]

# input file containing the reaction mechanism
reaction_mechanism = 'gri30.yaml'
# Resolution: The PFR will be simulated by 'n_steps' time steps or by a chai
# of 'n_steps' stirred reactors.
n_steps = 5000
### END INPUTS
```

In [816…
```
tau_H2_all = []
T_0_all = []
T_finals_H2_all = []
for j in pressures:
    T_finals_H2 = []
    tau_H2 = []
    for k, l in zip(Temperatures, lengths):
        pressure = j*ct.one_atm # constant pressure [Pa]
        T_0 = k
        # import the gas model and set the initial conditions
        gas1 = ct.Solution(reaction_mechanism)
        gas1.TPX = T_0, pressure, composition_0
        mass_flow_rate1 = u_0 * gas1.density * area

        # create a new reactor
        r1 = ct.IdealGasConstPressureReactor(gas1)
        # create a reactor network for performing time integration
        sim1 = ct.ReactorNet([r1])

        # approximate a time step to achieve a similar resolution as in the
        t_total = l / u_0
        dt = t_total / n_steps
        # define time, space, and other information vectors
        t1 = (np.arange(n_steps) + 1) * dt
        z1 = np.zeros_like(t1)
        u1 = np.zeros_like(t1)
```

```
        states1 = ct.SolutionArray(r1.thermo)
        for n1, t_i in enumerate(t1):
            # perform time integration
            sim1.advance(t_i)
            # compute velocity and transform into space
            u1[n1] = mass_flow_rate1 / area / r1.thermo.density
            z1[n1] = z1[n1 - 1] + u1[n1] * dt
            states1.append(r1.thermo.state)

        dTdt= np.gradient(states1.T, z1/u_0)
        maxidx=np.argmax(dTdt)
        tau_H2.append(z1[maxidx]/u_0-(T_0-states1.T[maxidx])/dTdt[maxidx])
        #point slope to solve for tau.
        #tau= t1 - (T_0-T(maxidx)/dT/dt(maxidx)
    for k in Temperatures:
        pressure = j * ct.one_atm
        T_0 = k
        fixed_length = 25  # constant length

        gas1 = ct.Solution(reaction_mechanism)
        gas1.TPX = T_0, pressure, composition_0
        mass_flow_rate1 = u_0 * gas1.density * area

        r1 = ct.IdealGasConstPressureReactor(gas1)
        sim1 = ct.ReactorNet([r1])

        t_total = fixed_length / u_0
        dt = t_total / n_steps
        t1 = (np.arange(n_steps) + 1) * dt
        z1 = np.zeros_like(t1)
        u1 = np.zeros_like(t1)
        states1 = ct.SolutionArray(r1.thermo)

        for n1, t_i in enumerate(t1):
            sim1.advance(t_i)
            u1[n1] = mass_flow_rate1 / area / r1.thermo.density
            z1[n1] = z1[n1 - 1] + u1[n1] * dt
            states1.append(r1.thermo.state)

        T_finals_H2.append(states1.T[-1])  # get final temp from full 25 m r

    print(f"\n Results at {j} atm:")
    print(f"{'T_0 [K]':>10} {'τ [s]':>15} {'ln(τ)':>15} {'T_f [K]':>15}")
    print("-" * 60)
    for T0, tau, log_tau, Tf in zip(Temperatures, tau_H2, np.log(tau_H2), T_
        print(f"{T0:>10} {tau:>15.5f} {log_tau:>15.5f} {Tf:>15.5f}")

    tau_H2_all.append(tau_H2)
    T_0_all.append(Temperatures)
    T_finals_H2_all.append(T_finals_H2)
```

```
Results at 1 atm:
  T_0 [K]           τ [s]           ln(τ)         T_f [K]
-------------------------------------------------------------
      750      2019.02145         7.61037       824.34692
     1000         0.00892        -4.71979      1075.38489
     1250         0.00111        -6.80536      1323.27666
     1500         0.00039        -7.84594      1571.57789
     1750         0.00018        -8.59933      1819.44087
     2000         0.00010        -9.18483      2064.91688

Results at 5 atm:
  T_0 [K]           τ [s]           ln(τ)         T_f [K]
-------------------------------------------------------------
      750       426.37203         6.05531       828.46083
     1000         0.13366        -2.01246      1075.38802
     1250         0.00029        -8.14173      1323.28937
     1500         0.00008        -9.37296      1571.64314
     1750         0.00004       -10.11255      1819.88805
     2000         0.00002       -10.68715      2066.82019

Results at 10 atm:
  T_0 [K]           τ [s]           ln(τ)         T_f [K]
-------------------------------------------------------------
      750       247.71219         5.51227       828.67345
     1000         0.09278        -2.37749      1075.38829
     1250         0.00026        -8.25072      1323.29529
     1500         0.00005       -10.00671      1571.66388
     1750         0.00002       -10.76896      1820.02081
     2000         0.00001       -11.32046      2067.37213
```

In [ ]:

```
plt.figure(figsize=(5, 4))

# Plot for each pressure
for i, j in enumerate(pressures):
    plt.plot(1000 / np.array(T_0_all[i]), np.log(tau_H2_all[i]), label=f'{j}

# Finalize plot
plt.xlabel(r'$1000/T_i\text{ }\mathrm{[1/K]}$')
plt.ylabel(r"$\ln(\tau)\text{ }\mathrm{[s]}$")
plt.legend(loc='upper left')
plt.grid(True)
plt.title('Ignition Delay Time vs. Initial Temperature;\n $H_2-O_2$ Reaction
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig('h2_igtime.png')
plt.show()
```

In [ ]:

```
markers = ['o', 's', '^']
linestyles = ['--', ':', '-.']
colors = ['tab:blue', 'tab:orange', 'tab:green']

plt.figure(figsize=(5, 4))
for i, j in enumerate(pressures):
    plt.plot(T_0_all[i], T_finals_H2_all[i],
             ls=linestyles[i], marker=markers[i], color=colors[i], label=f'{
```

```
plt.xlabel(r'$T_0$ [K]')
plt.ylabel(r'$T_f$ [K]')
plt.title('Final Temperature vs. Inlet Temperature;\n $H_2-O_2$ Reaction')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.savefig('h2_Tin_v_Tout.png')
plt.show()
```

CH4-O2 Simulation

In [810…

```
### INPUTS
composition_0 = 'CH4:0.01, O2:0.02, N2:0.970' #1% vol fuel, phi =1, rest N2
pressures = [1,5,10] # constant pressure [Pa]
lengths = [25, 1.5e-1, 1.5e-2, 1.5e-4, 1.5e-5, 1.5e-5] #PFR Length
Temperatures=[750, 1000, 1250, 1500, 1750, 2000] # inlet temperature [K]
u_0 = .006  # inflow velocity [m/s]
area = 1.e-4  # cross-sectional area [m**2]

# input file containing the reaction mechanism
reaction_mechanism = 'gri30.yaml'
# Resolution: The PFR will be simulated by 'n_steps' time steps or by a chai
# of 'n_steps' stirred reactors.
n_steps = 5000
### END INPUTS
```

In [811…

```
tau_CH4_all = []
T_0_CH4_all = []
T_finals_CH4_all = []
for j in pressures:
    T_finals_CH4 = []
    tau_CH4 = []
    for k, l in zip(Temperatures, lengths):
        pressure = j*ct.one_atm # constant pressure [Pa]
        T_0 = k
        # import the gas model and set the initial conditions
        gas1 = ct.Solution(reaction_mechanism)
        gas1.TPX = T_0, pressure, composition_0
        mass_flow_rate1 = u_0 * gas1.density * area

        # create a new reactor
        r1 = ct.IdealGasConstPressureReactor(gas1)
        # create a reactor network for performing time integration
        sim1 = ct.ReactorNet([r1])

        # approximate a time step to achieve a similar resolution as in the
        t_total = l / u_0
        dt = t_total / n_steps
        # define time, space, and other information vectors
        t1 = (np.arange(n_steps) + 1) * dt
        z1 = np.zeros_like(t1)
        u1 = np.zeros_like(t1)
        states1 = ct.SolutionArray(r1.thermo)
        for n1, t_i in enumerate(t1):
```

```
            # perform time integration
            sim1.advance(t_i)
            # compute velocity and transform into space
            u1[n1] = mass_flow_rate1 / area / r1.thermo.density
            z1[n1] = z1[n1 - 1] + u1[n1] * dt
            states1.append(r1.thermo.state)

        dTdt= np.gradient(states1.T, z1/u_0)
        maxidx=np.argmax(dTdt)
        tau_CH4.append(z1[maxidx]/u_0-(T_0-states1.T[maxidx])/dTdt[maxidx])
        #point slope to solve for tau.
        #tau= t1 - (T_0-T(maxidx)/dT/dt(maxidx)
    for k in Temperatures:
        pressure = j * ct.one_atm
        T_0 = k
        fixed_length = 25  # constant length

        gas1 = ct.Solution(reaction_mechanism)
        gas1.TPX = T_0, pressure, composition_0
        mass_flow_rate1 = u_0 * gas1.density * area

        r1 = ct.IdealGasConstPressureReactor(gas1)
        sim1 = ct.ReactorNet([r1])

        t_total = fixed_length / u_0
        dt = t_total / n_steps
        t1 = (np.arange(n_steps) + 1) * dt
        z1 = np.zeros_like(t1)
        u1 = np.zeros_like(t1)
        states1 = ct.SolutionArray(r1.thermo)

        for n1, t_i in enumerate(t1):
            sim1.advance(t_i)
            u1[n1] = mass_flow_rate1 / area / r1.thermo.density
            z1[n1] = z1[n1 - 1] + u1[n1] * dt
            states1.append(r1.thermo.state)

        T_finals_CH4.append(states1.T[-1])  # get final temp from full 25 m


    print(f"\n Results at {j} atm:")
    print(f"{'T_0 [K]':>10} {'τ [s]':>15} {'ln(τ)':>15} {'T_f [K]':>15}")
    print("-" * 60)
    for T0, tau, log_tau, Tf in zip(Temperatures, tau_CH4, np.log(tau_CH4),
        print(f"{T0:>10} {tau:>15.5f} {log_tau:>15.5f} {Tf:>15.5f}")
    tau_CH4_all.append(tau_CH4)
    T_0_CH4_all.append(Temperatures)
    T_finals_CH4_all.append(T_finals_CH4)
```

```
Results at 1 atm:
  T_0 [K]           τ [s]           ln(τ)         T_f [K]
-------------------------------------------------------------
      750      3154.67020         8.05664       997.81888
     1000        15.64284         2.75001      1237.40337
     1250         0.37602        -0.97810      1480.55137
     1500         0.01043        -4.56346      1724.43411
     1750         0.00087        -7.04422      1965.09470
     2000         0.00019        -8.57086      2195.96405

Results at 5 atm:
  T_0 [K]           τ [s]           ln(τ)         T_f [K]
-------------------------------------------------------------
      750       602.81311         6.40161       997.82500
     1000         2.16443         0.77216      1237.41281
     1250         0.09057        -2.40166      1480.63140
     1500         0.00397        -5.52971      1725.07275
     1750         0.00027        -8.21224      1968.04907
     2000         0.00005        -9.93578      2204.94872

Results at 10 atm:
  T_0 [K]           τ [s]           ln(τ)         T_f [K]
-------------------------------------------------------------
      750       310.46385         5.73807       997.83041
     1000         0.91157        -0.09258      1237.41430
     1250         0.04309        -3.14450      1480.65381
     1500         0.00253        -5.97958      1725.26380
     1750         0.00018        -8.61292      1968.94407
     2000         0.00003       -10.45154      2207.75238
```

In [ ]:

```
# ln(tau) vs 1000/T0
plt.figure(figsize=(5, 4))
for i, j in enumerate(pressures):
    plt.plot(1000 / np.array(T_0_CH4_all[i]), np.log(tau_CH4_all[i]), label=

plt.xlabel(r'$1000/T_0\text{ }\mathrm{[1/K]}$')
plt.ylabel(r"$\ln(\tau)\text{ }\mathrm{[s]}$")
plt.title('Ignition Delay Time vs. Inlet Temperature;\n $CH_4-O_2$ Reaction'
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.savefig('ch4_igtime.png')
plt.show()
```

In [ ]:

```
markers = ['o', 's', '^']
linestyles = ['--', ':', '-.']
colors = ['tab:blue', 'tab:orange', 'tab:green']

plt.figure(figsize=(5, 4))
for i, j in enumerate(pressures):
    plt.plot(T_0_CH4_all[i], T_finals_CH4_all[i],
             ls=linestyles[i], marker=markers[i], color=colors[i], label=f'{

plt.xlabel(r'$T_0$ [K]')
plt.ylabel(r'$T_f$ [K]')
plt.title('Final Temperature vs. Inlet Temperature;\n $CH_4-O_2$ Reaction')
```

```
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.savefig('ch4_Tin_v_Tout.png')
plt.show()
```

Temperature Profile Plot (Debugger)

In [826…

```
# INPUTS
composition_0 = 'H2:0.01, O2:0.005, N2:0.985'  # 1% vol fuel, phi =1, rest N
length = 0.00005 # PFR Length
T_0 = 1500  # Inlet temperature [K]
u_0 = 0.006  # Inflow velocity [m/s]
area = 1e-4  # Cross-sectional area [m^2]
reaction_mechanism = 'gri30.yaml'
n_steps = 5000

plt.figure(figsize=(5, 4))

for j in [1,5,10]:
    pressure = j * ct.one_atm  # Pressure in Pa
    gas = ct.Solution(reaction_mechanism)
    gas.TPX = T_0, pressure, composition_0

    mass_flow_rate = u_0 * gas.density * area
    r = ct.IdealGasConstPressureReactor(gas)
    sim = ct.ReactorNet([r])

    t_total = length / u_0
    dt = t_total / n_steps
    t = (np.arange(n_steps) + 1) * dt
    z = np.zeros_like(t)
    u = np.zeros_like(t)
    states = ct.SolutionArray(gas)

    for n, t_i in enumerate(t):
        sim.advance(t_i)
        u[n] = mass_flow_rate / area / r.thermo.density
        z[n] = z[n - 1] + u[n] * dt if n > 0 else 0
        states.append(r.thermo.state)

    plt.plot(z, states.T, label=f'{j} atm')
    dTdt= np.gradient(states1.T, z1/u_0)
    maxidx=np.argmax(dTdt)
    tau_H2.append(z1[maxidx]/u_0-(T_0-states1.T[maxidx])/dTdt[maxidx])
# Final plot settings
plt.xlabel('$z$ [m]')
plt.ylabel('Temperature [K]')
plt.title('Temperature Profiles at Different Pressures')
plt.legend(title='$T_0$ = 1500 K,\nInitial Pressure: ')
plt.grid(True)
plt.tight_layout()
plt.savefig('tempvspres.png')
plt.show()
```

```
<Figure size 500x400 with 0 Axes>
```

In [821…

```
print(ct.__version__)
```

```
3.1.0
```

In [ ]: